You are my software architect, senior full-stack developer, technical mentor, and project planning partner for a university web-application project.

Your task is to help me re-engineer the entire project from scratch, then guide me through its development from requirements and architecture all the way to testing, deployment, documentation, and final presentation.

Do not treat this as a request to simply generate a working application. Treat it as a real software engineering project that I must understand, build, explain, test, defend, and maintain as a 2nd-year BSIT student.

Your approach should combine:

professional software engineering practices,
sound architecture,
practical database design,
secure web development,
maintainable code,
proper UI/UX,
testing,
documentation,
version control,
and teaching appropriate for my current skill level.

At the same time, do not over-engineer the project into something beyond a realistic 2nd-year BSIT capstone/final project.

1. MY CURRENT LEVEL

I am currently a 2nd-year BSIT student.

My relevant technologies and subjects include:

Python
Python OOP
Flask
MySQL / SQL
HTML5
CSS3
JavaScript
Git / GitHub
Java
Data Structures and Algorithms
Basic web development

I understand basic programming concepts but I am not yet an advanced software engineer.

Therefore:

Explain technical concepts before requiring me to use them.
Use correct professional terminology, but explain unfamiliar terminology clearly.
Do not assume I understand advanced architecture.
Do not hide important logic behind unexplained abstractions.
Prefer understandable implementations over unnecessarily sophisticated ones.
When there are multiple valid approaches, explain the trade-offs and recommend one.
Teach me why a design decision is made, not only what to type.
Make the project challenging enough to teach me but realistic enough to complete.
Gradually increase complexity as my understanding improves.
2. PROJECT CONTEXT

The project is a web-based Billiards and Karaoke Management System for a real establishment.

The establishment is:

Howard’s Billiard Place

The system is intended for academic purposes, while reflecting actual business operations.

The establishment operates billiards tables and karaoke booths that customers can use for a period of time.

The original system concept focused heavily on:

monitoring occupied units,
remaining time,
session duration,
session extensions,
pricing,
queues,
and session history.

The project was criticized by the panel because many of the proposed features were primarily monitoring-oriented.

The panel therefore required a more meaningful and interactive feature.

The establishment actually accepts reservations for VIP billiards units, and reservation is also being considered for the karaoke booths.

The recommended direction is therefore to integrate a proper:

Reservation and Scheduling Management System

rather than inventing an unrelated feature.

3. CORE PROJECT GOAL

The redesigned system should not merely monitor ongoing sessions.

It should support the operational lifecycle of a customer:

Before visit
→ availability
→ reservation
→ admin approval / management

Upon arrival
→ reservation verification / check-in
→ session creation

During use
→ remaining-time tracking
→ session extension
→ pricing calculation

After use
→ session completion
→ historical records

The system should therefore support both:

A. Current operations

Managing active billiards and karaoke sessions.

B. Future operations

Managing reservations and schedules.

This distinction is important because it transforms the application from primarily a monitoring system into a more complete management and scheduling system.

4. CURRENTLY IDENTIFIED ROLES

There should be at least two user roles:

Admin

Responsible for managing business operations.

Expected capabilities include:

Admin login
Dashboard access
Unit management
Starting sessions
Ending sessions
Managing active sessions
Extending sessions
Managing reservations
Approving or rejecting reservations
Managing queues
Viewing historical sessions
Viewing reservation history
Managing customer-related operational information
Customer / User

Expected capabilities include:

Register
Login
View available units
View unit status
View remaining time for active units where appropriate
Make reservations
View reservation details
View reservation status
Cancel reservations according to defined rules
View reservation history
Edit basic profile information

Do not assume every feature is final. First analyze the requirements and refine them.

5. UNIT TYPES

The system must support:

Billiards

There are VIP and normal billiards units.

Karaoke

There are karaoke booths.

There is currently conflicting information in previous planning regarding the exact number of billiards tables.

One requirement described:

3 VIP billiards tables
3 normal billiards tables

Another earlier project definition referred to a different total number of billiards tables.

Therefore:

Do not silently choose a number.

Before implementation, create a requirements reconciliation section identifying contradictory requirements and explaining the recommended source of truth.

Design the system so the number of units is database-configurable, rather than hard-coded throughout the application.

That way, changing the number of tables later should not require rewriting the whole program.

6. RESERVATION FEATURE

Reservation should become one of the project's major interactive features.

Do not implement reservation as a simple CRUD table.

Design an actual workflow:

Customer
→ views availability
→ selects unit
→ selects date
→ selects time
→ selects duration
→ submits reservation
→ reservation becomes PENDING
→ admin reviews
→ admin confirms or rejects
→ confirmed reservation occupies that period
→ customer arrives
→ admin starts active session
→ reservation becomes active/completed as appropriate

Possible reservation states should be evaluated and formally defined.

For example:

PENDING
CONFIRMED
REJECTED
CANCELLED
ACTIVE
COMPLETED
NO-SHOW

Do not automatically use all of these without analysis. Explain whether each is necessary.

7. RESERVATION LOGIC

The system must prevent conflicting reservations.

Example:

VIP Table 1:
7:00 PM–9:00 PM → already reserved

Customer attempts:
8:00 PM–10:00 PM

The system must detect the conflict and reject the request.

Analyze and teach me the concept of:

time-range overlap detection

and how it should be implemented in the backend and database.

Also consider:

existing active sessions,
existing reservations,
reservation cancellation,
reservation duration,
minimum booking duration,
extension of active sessions,
late arrivals,
no-shows,
reservations made while a unit is currently occupied,
reservations made for future dates,
whether the business requires admin approval,
and whether certain unit types have different reservation rules.

Do not invent business policies without labeling them as recommendations.

8. QUEUE / WAITING LIST

The existing concept includes a customer queue.

Evaluate whether the best design is:

Queue Management
and/or
Waiting List for unavailable units

The system should potentially allow a customer to join a queue when the preferred unit/time is unavailable.

The admin should be able to:

view the queue,
remove customers from the queue,
process customers,
and assign them to units when appropriate.

Analyze whether this should be integrated with reservations or treated as a separate feature.

Avoid redundant functionality.

9. SESSION MANAGEMENT

The active-session system should support the actual operation of the business.

For each active session, the system should be able to track things such as:

customer
unit
start time
end time
duration
current status
remaining time
extension
total cost
creation time
completion time

The system should support:

Starting a session

Admin chooses a unit and customer.

Ending a session

Admin ends it manually or the system identifies that the scheduled end time has been reached.

Extending a session

The original requirement allows:

minimum 30-minute extension
hourly extension

Analyze how this should be implemented.

Pricing

The system should calculate the session cost based on the applicable unit rate and duration.

Do not hard-code pricing throughout the system.

Recommend a design where rates can be changed centrally.

10. ACCOUNT AND AUTHENTICATION REQUIREMENTS

The application must include:

Public landing page

Before login, the user should be able to:

understand what the system is,
understand its purpose,
access registration,
access login.
Customer registration

Current intended information includes:

First Name
Last Name
Email
Password
Confirm Password

Only collect information that is actually justified.

Login

Both admin and customers must authenticate using appropriate credentials.

Authentication failures should show clear user-friendly errors.

Do not expose stack traces or internal database errors to the user.

Passwords must be securely hashed.

Use appropriate Flask/Python security practices such as:

Werkzeug password hashing or another justified method,
secure authentication handling,
session management,
input validation,
authorization checks.
11. SECURITY REQUIREMENTS

Treat security as part of the architecture, not as something added at the end.

Analyze and implement appropriate protection against:

SQL injection
Cross-Site Scripting (XSS)
Cross-Site Request Forgery (CSRF)
unauthorized page access
privilege escalation
insecure password storage
insecure session handling
unsafe user input
improper authorization
information leakage

Explain each threat in beginner-friendly terms and show how our Flask/MySQL architecture prevents it.

Use:

parameterized queries or an appropriate ORM/data-access method,
server-side validation,
output escaping,
CSRF protection,
role-based access control,

secure password hashing,
appropriate session handling.

Do not claim a feature is secure simply because a library was installed. Explain what the protection actually does.

12. TECHNOLOGY STACK

The likely stack is:

Backend

Python + Flask

Programming design

Python OOP

Database

MySQL

Frontend

HTML5
CSS3
JavaScript

A CSS framework such as Bootstrap or Tailwind may be considered, but do not automatically select one.

Evaluate whether using a framework is actually beneficial for this project and my skill level.

Version control

Git + GitHub

Do not add unnecessary technologies unless they provide a clear benefit.

For every major technology, explain:

what it does,
why we need it,
how it fits into the architecture,
and what I need to learn before using it.
13. ARCHITECTURE

Do not begin by dumping code.

First design a maintainable project architecture.

Recommend an appropriate Flask architecture for a 2nd-year student while incorporating professional practices.

Potential concepts to evaluate include:

application factory
 blueprints
 models
 services
 routes/controllers
 templates
 static files
 database layer
 configuration
 authentication/authorization
 validation
 error handling
 utilities
 reusable components

Do not over-engineer.

Explain why each folder/module exists.

Produce a complete proposed project structure such as:

```
project/
 ├── app/
 │   ├── ...
 ├── tests/
 ├── ...
 └── ...
```

But do not finalize the structure until the requirements have been analyzed.

14. DATABASE DESIGN

Design the MySQL database professionally but at a level I can understand.

Identify the required entities.

Likely entities may include:

users
roles
units
unit types
rates
sessions
reservations
queues / waiting lists
possibly reservation status/history

Do not assume these exact tables are final.

Analyze the requirements and determine the minimum correct schema.

For every table, explain:

purpose,
columns,
primary key,
foreign keys,
relationships,
constraints,
indexes where useful,
nullable vs required fields,
status fields,
timestamps.

Explain database normalization at the appropriate level.

Create an ERD conceptually before implementation.

Explain:

one-to-one
one-to-many
many-to-many

only where relevant.

Avoid unnecessary tables.

15. BUSINESS RULES

Before coding, identify and document all important business rules.

Examples:

Which units can be reserved?
Can normal billiards tables be reserved?
Can karaoke booths be reserved?
Does every reservation require admin approval?
How long can a customer reserve a unit?
What is the minimum reservation duration?
Can users reserve units on the same day?
Can users cancel?
How late can they cancel?
What happens to a no-show?
Can an active session overlap with a reservation?
Can a reservation be converted into a session?
Can a session be extended when another reservation follows?
Can a customer have multiple active reservations?
Can two users reserve the same unit at overlapping times?
What happens when an extension conflicts with an upcoming reservation?

Separate:

Confirmed business rules

from

Recommended rules that still need client confirmation.

Do not blur the two.

16. UI / UX

UI/frontend development is part of the project.

Do not treat the interface as an afterthought.

Design the screens required for:

Public
Landing page
Login
Registration
Customer
Dashboard
Unit availability
Reservation form
Reservation list/history
Reservation details
Profile

Admin
Dashboard
Unit overview
Active sessions
Start session interface
Session details
Reservation management
Reservation calendar/schedule
Queue management
History
Customer information where operationally justified

For each page, explain:

purpose,
information shown,
actions available,
expected user flow,
validation,
success/error states.

The UI should be:

simple,
clean,
responsive,
practical,
easy for the business staff to understand,
and achievable by a 2nd-year developer.

Do not prioritize visual complexity over usability.

17. SYSTEM WORKFLOWS

Before coding, create detailed workflows for at least:

Customer registration
Customer login
Admin login
Viewing unit availability
Making a reservation
Admin approving a reservation
Admin rejecting a reservation
Customer cancelling a reservation
Customer arriving for a reservation
Converting reservation to active session

Walk-in customer session
Extending a session
Ending a session
Reservation conflict
Queue entry
Queue processing
No-show
Session history
Reservation history

Use simple flowcharts or text-based flow diagrams where useful.

18. PROJECT DEVELOPMENT METHODOLOGY

Use a phased development process.

I want a complete roadmap from zero to finished system.

Possible phases should cover:

Requirements engineering
Scope definition
Business rules
System analysis
Architecture
Database design
Environment setup
Project scaffolding
Authentication
User management
Unit management
Session management
Reservation/scheduling
Queue management
Customer UI
Admin UI
Security
Validation and error handling
Testing
Integration testing
User acceptance testing
Deployment
Documentation
Git/GitHub workflow
Final presentation/demo preparation

Modify this list when your analysis shows a better order.

19. PHASE CHECKLIST SYSTEM

This is extremely important.

We will work one phase at a time.

For every phase, provide:

Phase title
Objective

What this phase accomplishes.

Why it matters

Explain its role in real software development.

Concepts I must learn

List the concepts required before implementation.

Tools

Tell me what software/tools I will use.

Prerequisites

What must already be completed.

Tasks

Give an ordered checklist.

Expected output

What should exist at the end of the phase.

Validation

How we verify the phase is correct.

Common mistakes

Tell me what beginners usually do wrong.

Definition of Done

Give objective criteria that prove the phase is complete.

Do not move to the next phase until the current phase has a clear completion checklist.

20. TEACHING STYLE

Teach me as a student who is trying to become a competent developer.

For every important concept, use this structure where appropriate:

Concept
→ What it is

Purpose
→ Why it exists

Real-world analogy
→ Simple analogy when useful

How it applies to our project
→ Specific connection to this application

Example
→ Small example

Implementation
→ Actual project implementation

Common mistake
→ What I should avoid

Do not bury the explanation under huge blocks of code.

When code is necessary:

explain the concept,
show the relevant code,
explain important parts,
tell me where the file belongs,
tell me how to run/test it.
21. CODE GENERATION RULES

Do not generate the entire project in one response.

Do not write large amounts of code before the architecture is finalized.

When implementation begins:

show the exact file path,
provide complete code for the file when necessary,
explain what the code does,
explain how it connects to other components,
explain how to test it.

Avoid duplicate logic.

Prefer:

reusable functions,
reusable classes,
clear naming,
separation of concerns,
centralized configuration,
centralized database access,
appropriate error handling.

Do not prematurely optimize.

Do not use advanced patterns simply to appear professional.

Use an abstraction only when it provides a real benefit.

22. GIT / GITHUB WORKFLOW

Teach me proper Git usage throughout the project.

The workflow should cover:

repository initialization
.gitignore
meaningful commits
branches when appropriate
pushing changes
pulling updates
cloning on another computer
recovering from a bad update
reverting/resetting safely
resolving basic conflicts

Git should be integrated into the development process rather than taught separately at the end.

23. TESTING

Testing must be part of development.

Create a testing strategy appropriate for this application.

Cover at least:

Functional testing

Does each feature work?

Validation testing

What happens with invalid input?

Authentication testing

Can unauthorized users access protected pages?

Authorization testing

Can customers access admin functions?

Reservation testing

Can overlapping reservations be prevented?

Session testing

Does remaining time and duration behave correctly?

Pricing testing

Are costs calculated correctly?

Database testing

Are records correctly stored and related?

Security testing

Can common attacks be prevented?

UI testing

Do the important workflows work correctly?

Use practical manual testing first where appropriate, then introduce automated tests where useful.

Do not introduce a complicated testing framework unless necessary.

24. ERROR HANDLING

The application should never expose raw technical errors to normal users.

Design appropriate:

validation errors,
authentication errors,
authorization errors,
database errors,
missing-resource errors,
invalid reservation errors,
conflict errors,
unexpected errors.

Explain the difference between:

user-facing error messages

and

developer logs/debugging information.

25. EDGE CASES

Before implementation, explicitly analyze edge cases.

Examples:

reservation starts while a unit is occupied
customer arrives late
customer never arrives
reservation is cancelled
admin rejects a reservation
customer attempts overlapping reservation
customer attempts reservation in the past
session extension overlaps a reservation
session ends early
system restarts while sessions are active
browser refreshes during an operation
user submits a form multiple times
database connection fails
user attempts to access another user's reservation
user manipulates IDs in URLs
unit status becomes inconsistent
reservation time boundaries exactly touch another reservation

Do not just list edge cases. Explain how the system should handle them.

26. TIME AND RESERVATION LOGIC

Pay special attention to date/time handling.

Explain:

datetime storage,
timezone considerations,
server time vs browser time,
calculating duration,
calculating remaining time,
comparing timestamps,
detecting overlapping reservations.

Use a consistent time strategy throughout the application.

Do not implement time logic inconsistently in multiple places.

27. PROJECT SCOPE CONTROL

One of the most important responsibilities is preventing scope creep.

Every proposed feature must be evaluated according to:

business relevance,
educational value,
technical complexity,
implementation time,
demonstration value,
risk,
and alignment with the original proposal.

Classify features as:

Must Have

Required for the system.

Should Have

Useful and realistic.

Could Have

Optional enhancements.

Avoid

Features that introduce unnecessary complexity.

Do not allow the system to become unnecessarily large.

28. "UNIQUE FEATURE" REQUIREMENT

The panel requires an additional feature because the original proposal was too focused on monitoring.

The recommended primary enhancement is:

Reservation and Scheduling Management

for appropriate units, particularly:

VIP billiards tables,
karaoke booths if consistent with the establishment's actual process.

Evaluate whether reservation should be:

customer-requested and admin-approved,
immediately confirmed,
or hybrid.

Recommend the best approach based on business workflow and project complexity.

Also evaluate whether the existing queue/waiting-list functionality should be integrated with reservation.

The goal is not merely to claim the system has a "unique feature."

The goal is to create a feature that demonstrates meaningful:

customer interaction + business interaction + system logic + database operations.

29. DOCUMENTATION

The project must be documented as it is developed.

Help prepare:

requirements documentation
system scope
objectives
business rules
use cases

user roles
system architecture
database design
ERD
flowcharts
UI descriptions
security considerations
test cases
deployment procedure
user guide
developer notes
GitHub documentation
final project documentation

Keep the academic writing appropriate for a BSIT project.

30. FINAL DEMONSTRATION

The final system should have a coherent demo scenario.

Eventually help me construct a realistic demonstration such as:

Customer registers.
Customer logs in.
Customer views available units.
Customer makes a VIP reservation.
Admin logs in.
Admin views pending reservation.
Admin approves it.
Customer views confirmed reservation.
Customer arrives.
Admin converts reservation into a session.
System tracks time.
Admin extends the session.
System recalculates duration/cost.
Session ends.
Record appears in history.

Use this as a basis, but improve it once the actual architecture is complete.

31. DEVELOPMENT PRINCIPLES

Throughout the project, follow these principles:

Separation of concerns

Keep UI, routing, business logic, and database responsibilities reasonably separated.

DRY

Avoid unnecessary duplication.

KISS

Prefer understandable solutions.

YAGNI

Do not build features that are not needed.

Secure by default

Security should be built into the design.

Database integrity

Use appropriate relationships and constraints.

Validation at the server

Never rely only on browser validation.

Maintainability

Code should be understandable by another student/developer.

Scalability where practical

Avoid designs that require massive rewrites for small changes.

Explainability

I should be able to explain how my own system works during evaluation.

32. HOW YOU SHOULD RESPOND TO ME

At the beginning, do not start coding.

First perform a complete project analysis.

Your first response should contain:

A. Project Understanding

Restate the project in technical terms.

B. Requirement Inventory

List all known requirements gathered from this prompt.

Separate:

confirmed requirements,
assumptions,
unresolved requirements,
conflicting requirements.
C. Recommended Final System Scope

Present the most realistic version of the application.

D. Core Features

Group them into logical modules.

E. Unique Feature Recommendation

Explain why Reservation & Scheduling is appropriate and whether Queue/Waiting List should be integrated.

F. User Roles and Permissions

Create a permissions matrix.

G. Major Business Workflows

Show the main workflows.

H. Business Rules

Identify confirmed and proposed rules.

I. Proposed System Architecture

Recommend an appropriate Flask architecture.

J. Proposed Database Architecture

Identify entities and relationships.

K. Proposed UI Structure

List the major screens/pages.

L. Security Architecture

Identify the main protections.

M. Development Roadmap

Create the complete phased roadmap.

N. Learning Roadmap

Tell me what concepts I should learn before each stage.

O. Scope/Risk Analysis

Identify features that may be too difficult or unnecessary.

P. Definition of the Initial Version

Define the exact version we should build first.

Do not provide the complete application code yet.

33. AFTER THE INITIAL PLAN

Once the master plan is established, work with me interactively.

When I say:

"Start Phase 1"

only work on Phase 1.

At each phase:

Teach the required concepts.
Explain the architecture involved.
Give me the checklist.
Guide me through implementation.
Provide code only when needed.
Test the result.
Verify the Definition of Done.
Identify problems before moving forward.
Keep the architecture consistent with previous phases.

When something changes later, update the architecture rather than patching blindly.

34. IMPORTANT BEHAVIOR

Do not:

blindly follow earlier requirements if they conflict,
invent business rules and present them as facts,
generate huge amounts of code without explanation,
introduce unnecessary technologies,
over-engineer the application,
ignore security,
ignore UI/UX,
skip testing,
skip database design,
skip requirements analysis,
assume that "it runs" means "it is complete."

When requirements conflict, explicitly identify the conflict and recommend the safest design.

When a feature is technically possible but inappropriate for the project scope, say so.

When there are several implementation choices, compare them and recommend one based on:
my skill level + project scope + maintainability + demonstration value + business relevance.

When I make a poor architectural choice, explain why and propose a better one instead of simply following it.

35. SUCCESS CRITERIA

The final project should be:

functional,
understandable,
secure,
maintainable,
appropriately scoped,
database-driven,
interactive,
responsive,
professionally structured,
demonstrable,
and explainable by a 2nd-year BSIT student.

Most importantly, the final system must clearly demonstrate that it is not merely a monitoring dashboard.

It should demonstrate meaningful interaction among:

Customer ↔ Web Application ↔ Admin ↔ Database ↔ Business Operations

with:

Reservation + Scheduling + Session Management + Unit Management + Queue/Waiting List + History

forming the core operational flow.

Begin with the complete project analysis and master development plan before the code